# Report

# Insufficient Cryptography & Poor Code Quality

# Code Tampering & Reverse Engineering

**Suyash A Gunjal**

## Insufficient Cryptography:

Threat agents who exploit insecure cryptography in mobile applications can undermine the confidentiality, integrity, and authenticity of sensitive information. These threat agents include attackers who target cryptographic algorithms or implementations to decrypt sensitive data, malicious insiders who manipulate cryptographic processes or leak encryption keys, state-sponsored actors engaged in cryptanalysis for intelligence purposes, cybercriminals who exploit weak encryption to steal valuable data or conduct financial fraud, and attackers who leverage vulnerabilities in cryptographic protocols or libraries.

There are several ways in which insecure cryptography and insecure hash functions can manifest in a mobile application:

**Weak Encryption Algorithms**: The mobile app may use encryption algorithms that are known to be weak or vulnerable to attacks. These algorithms may have known weaknesses, be outdated, or lack the necessary level of security to protect sensitive data effectively.

**Insufficient Key Length:** Inadequate key length can weaken the encryption strength. If the mobile app uses short or easily guessable encryption keys, it becomes easier for attackers to decrypt the encrypted data through brute-force or other cryptographic attacks.

**Improper Key Management:** Poor key management practices, such as storing encryption keys insecurely or transmitting them in plain text, can expose the keys to unauthorized access. Attackers who gain access to the keys can decrypt the data without difficulty.

**Flawed Encryption Implementation:** The encryption/decryption process itself may be implemented incorrectly or contain programming flaws. These implementation errors can introduce vulnerabilities that attackers can exploit to bypass or weaken the encryption protections.

To prevent “insufficient cryptography” vulnerabilities in the mobile application, consider the following best practices:

**Use Strong Encryption Algorithms**: Implement widely accepted and secure encryption algorithms, such as AES (Advanced Encryption Standard), RSA

(Rivest-Shamir-Adleman), or Elliptic Curve Cryptography (ECC). Stay updated with current cryptographic standards and avoid deprecated or weak algorithms.

**Ensure Sufficient Key Length**: Select encryption keys with an appropriate length to ensure strong cryptographic strength. Follow industry recommendations for key lengths, considering the specific encryption algorithm being used.

**Follow Secure Key Management Practices**: Employ secure key management techniques, such as using key vaults or hardware security modules (HSMs) to securely store encryption keys. Protect keys from unauthorized access, including restricting access to authorized personnel, encrypting keys at rest, and using secure key distribution mechanisms.

## POOR CODE QUALITY

Poor Code Quality means the software is written with bad programming practices that make the application unstable, hard to maintain, and insecure. It doesn't always look like a "direct vulnerability" at first, but it often becomes the root cause of security issues because the code behaves in unexpected ways and developers miss important protections.

Poor code quality doesn't usually become apparent until one of the following occurs:

- Progress on the project slows dramatically as it seems to near completion (and occasionally it never is completed)
- The software, once in use, is very unstable and crashes frequently
- The software works when you use it the "normal way" (e.g. what you may see in a demo), but can't handle "unusual" behavior
- There is a steady stream of bugs, even many months after the software has been released

There are many factors that affect codes readability.

- Poor or outdated names

- Dead code interspersed alongside living code
- Inconsistent formatting
- Placing related code in far apart from each other in the files
- Special hard coded cases

## CODE TEMPERING:

Code Tampering (sometimes written incorrectly as *code tempering*) is the act of modifying an application's original code or executable files without authorization, in order to change how the software behaves. The main goal of tampering is usually to bypass security controls, remove restrictions, manipulate app logic, or inject malicious behavior into the application.

### How Code Tampering Happens

Attackers typically perform code tampering using techniques such as:

- **Reverse engineering**: analyzing the app to locate security checks or payment logic
- **Binary patching**: directly editing the compiled app binaries or functions
- **Repackaging**: changing the code and rebuilding the application package
- **Hooking**: modifying function behavior at runtime (e.g., using Frida or Xposed)
- **Client-side request manipulation**: changing app behavior by altering request values and logic

### Why Code Tampering is Dangerous

Code tampering is a serious security issue because it breaks the trust that the software behaves correctly. When attackers change application logic, they can:

- bypass **authentication/authorization**
- unlock paid features for free

- manipulate **transactions** (prices, quantities, account IDs)
- steal sensitive data (tokens, credentials, user info)
- inject spyware or backdoors into the app
- damage the company's brand and cause financial loss

## REVERSE ENGINEERING

Reverse engineering is the process of evaluating the functionalities and information flow of a program or hardware in order to comprehend its functioning and behavior. It is done mostly to learn how something works and to analyze it, although it is frequently used to reproduce or improve the item. Reverse-engineering is a widespread practice in cyber defense to analyze malware. Software, physical devices, military technologies, and even biological processes including gene function can all be reverse engineered.

Reverse engineering is used to teach someone how something works, repurpose outmoded goods, do a security study, obtain a competitive advantage, or any number of other things, depending on the technology. Reverse engineering is the process of extracting information from a final product, regardless of how the knowledge is used or what it pertains to.

Security Impact:

Reverse engineering can lead to:

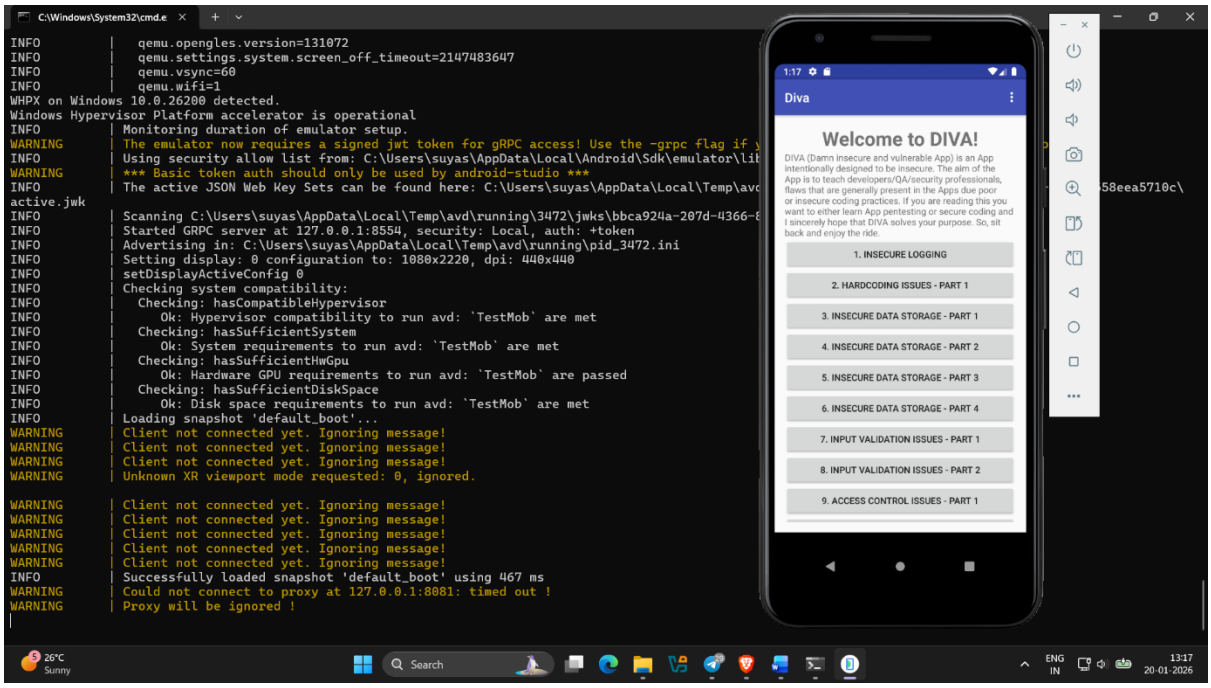
- exposure of sensitive info (keys, tokens, endpoints)
- bypass of app security features
- code tampering and repackaging
- creation of fake or malicious versions of apps

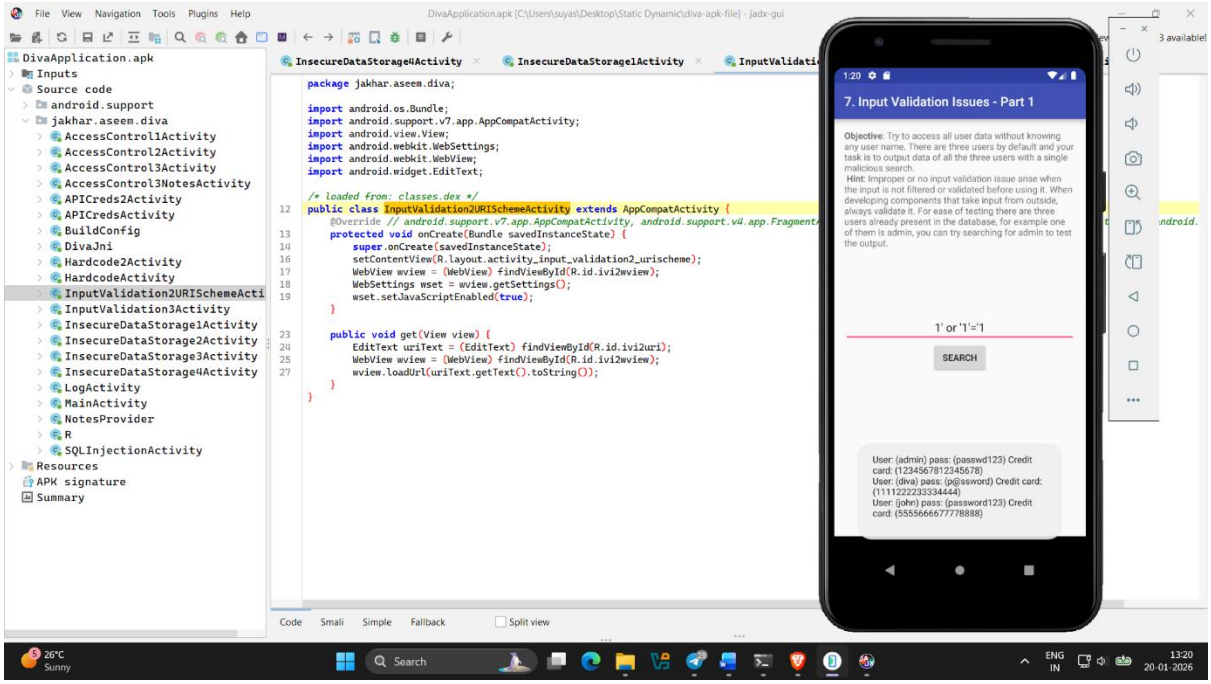
**How to Protect Against Reverse Engineering**

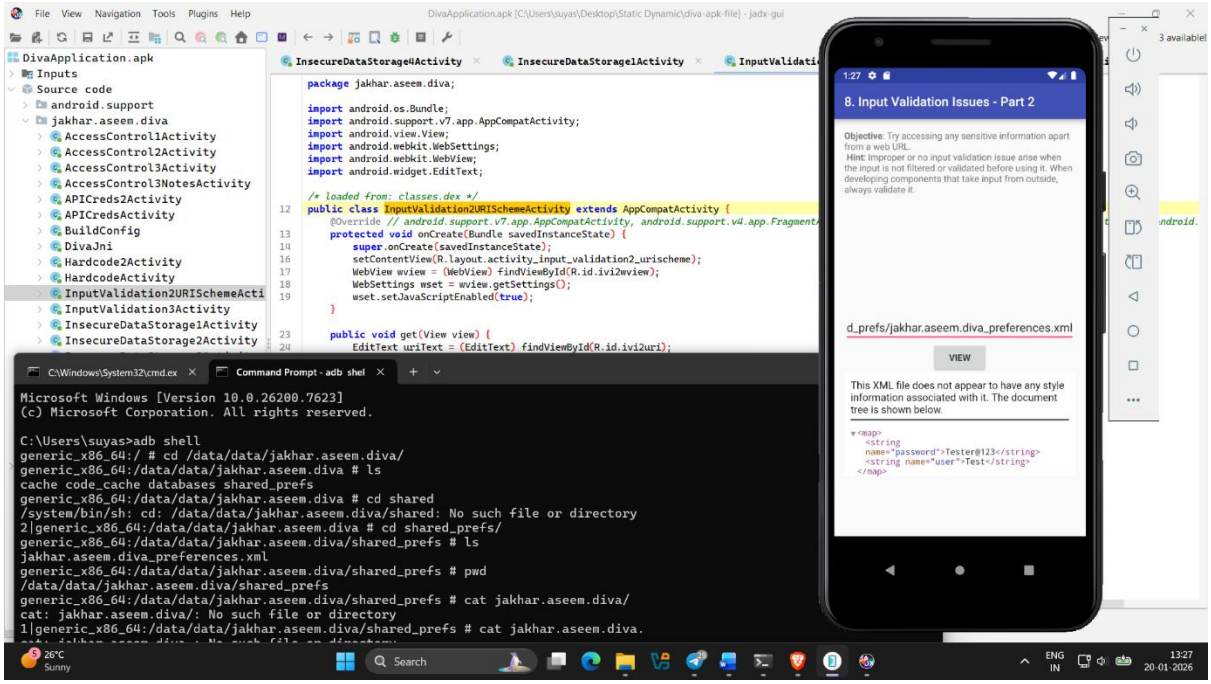
- Obfuscation (ProGuard/R8, DexGuard)

- Remove hardcoded secrets
- Server-side validation for security decisions
- Anti-debugging / anti-tamper checks
- Certificate pinning (with runtime protection)
- Code integrity checks and signing

## Implementation:




File View Navigation Tools Plugins Help
DivaApplication.apk [C:\Users\suyas\Desktop\Static Dynamic\diva-apk-file] - jadx-gui
DivaApplication.apk
Inputs
Source code
android.support
jakhar.aseem.diva
AccessControl1Activity
AccessControl2Activity
AccessControl3Activity
AccessControl3NotesActivity
APICreds2Activity
APICredsActivity
BuildConfig
DivaJni
Hardcode2Activity
HardcodeActivity
InputValidation2URISchemeActi
InputValidation3Activity
InsecureDataStorage1Activity
InsecureDataStorage2Activity
InsecureDataStorage3Activity
InsecureDataStorage4Activity
LogActivity
MainActivity
NotesProvider
R
SQLInjectionActivity
Resources
APK signature
Summary
InsecureDataStorage4Activity
InsecureDataStorage1Activity
package jakhar.aseem.diva;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.webkit.WebSettings;
import android.webkit.WebView;
import android.widget.EditText;
/* loaded from: classes.dex */
public class InputValidation2URISchemeActivity extends AppCompatActivity {
@Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.Fragment
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_input_validation2_urischeme);
WebView wview = (WebView) findViewById(R.id.ivi2wview);
WebSettings wset = wview.getSettings();
wset.setJavaScriptEnabled(true);
}
public void get(View view) {
EditText uriText = (EditText) findViewById(R.id.ivi2uri);
WebView wview = (WebView) findViewById(R.id.ivi2wview);
wview.loadUrl(uriText.getText().toString());
}
}
Code Smali Simple Fallback Split view
1:20
7. Input Validation Issues - Part 1
Objective: Try to access all user data without knowing any user name. There are three users by default and your task is to output data of all the three users with a single malicious search.
Hint: Improper or no input validation issue arise when the input is not filtered or validated before using it. When developing components that take input from outside, always validate it. For ease of testing there are three users already present in the database, for example one of them is admin, you can try searching for admin to test the output.
1' or '1'='1
SEARCH
User: (admin) pass: (passwd123) Credit card: (1234567812345678)
User: (diva) pass: (p@ssword) Credit card: (1111222233334444)
User: (john) pass: (password123) Credit card: (5555666677778888)
26°C Sunny
Search
ENG IN
13:20
20-01-2026


File View Navigation Tools Plugins Help
DivaApplication.apk [C:\Users\suyas\Desktop\Static Dynamic\diva-apk-file] - jadx-gui
DivaApplication.apk
Inputs
Source code
android.support
jakhar.aseem.diva
AccessControl1Activity
AccessControl2Activity
AccessControl3Activity
AccessControl3NotesActivity
APICreds2Activity
APICredsActivity
BuildConfig
DivaJni
Hardcode2Activity
HardcodeActivity
InputValidation2URISchemeActi
InputValidation3Activity
InsecureDataStorage1Activity
InsecureDataStorage2Activity
package jakhar.aseem.diva;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.webkit.WebSettings;
import android.webkit.WebView;
import android.widget.EditText;
/* loaded from: classes.dex */
public class InputValidation2URISchemeActivity extends AppCompatActivity {
C:\Windows\System32\cmd.ex
Command Prompt - adb shel
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.
C:\Users\suyas>adb shell
generic_x86_64:/ # cd /data/data/jakhar.aseem.diva/
generic_x86_64:/data/data/jakhar.aseem.diva # ls
cache code_cache databases shared_prefs
generic_x86_64:/data/data/jakhar.aseem.diva # cd shared
/system/bin/sh: cd: /data/data/jakhar.aseem.diva/shared: No such file or directory
2|generic_x86_64:/data/data/jakhar.aseem.diva # cd shared_prefs/
generic_x86_64:/data/data/jakhar.aseem.diva/shared_prefs # ls
jakhar.aseem.diva_preferences.xml
generic_x86_64:/data/data/jakhar.aseem.diva/shared_prefs # pwd
/data/data/jakhar.aseem.diva/shared_prefs
generic_x86_64:/data/data/jakhar.aseem.diva/shared_prefs # cat jakhar.aseem.diva/
cat: jakhar.aseem.diva/: No such file or directory
1|generic_x86_64:/data/data/jakhar.aseem.diva/shared_prefs # cat jakhar.aseem.diva.
1:27
8. Input Validation Issues - Part 2
Objective: Try accessing any sensitive information apart from a web URL.
Hint: Improper or no input validation issue arise when the input is not filtered or validated before using it. When developing components that take input from outside, always validate it.
d_prefs/jakhar.aseem.diva_preferences.xml
VIEW
This XML file does not appear to have any style information associated with it. The document tree is shown below.
<map>
<string name="password">Tester@123</string>
<string name="user">Test</string>
</map>
26°C Sunny
Search
ENG IN
13:27
20-01-2026

## References:

https://www.zenarmor.com/docs/network-security-tutorials/what-is-reverse-engineering

https://owasp.org/www-project-mobile-top-10/2016-risks/m7-client-code-quality

https://blog.codacy.com/cryptographic-failures-owasp-top-10